Apellidos: Serrano Tonconi Nombre: Geraldine Helen CI:6723815LP

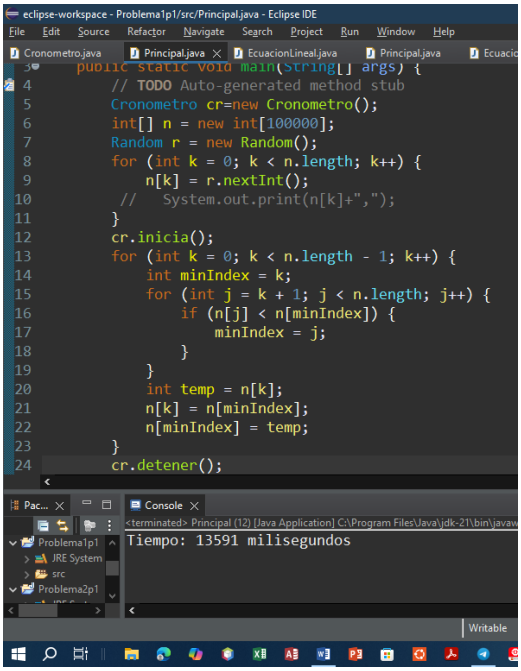
1. Cronometro. Diseñe una clase llamada Cronometro. La clase contiene:

a) Los atributos privados inicia y finaliza con métodos getter.
b) Un constructor sin argumentos que inicialice inicia con la hora actual.
c) Un método llamado inicia() que restablece inicia a la hora actual.
d) Un método llamado detener() que establece finaliza a la hora actual.
e) Un método llamado lapsoDeTiempo() que retorna el tiempo transcurrido del cronometro en milisegundos.

Dibuje el diagrama UML de la clase e impleméntela. Escriba un programa de prueba que mida el tiempo de ejecución de la ordenación de 100.000 números mediante la ordenación por selección.

Diagrama UML

Cronometro
- Inicia long - Finaliza long
+ Cronometro() + getInicia() + getFinaliza + inicia() + detener() + lapsoDeTiempo



Código

CLASE CRONOMETRO

```
public class Cronometro {

        private long inicia;

        private long finaliza;

        public Cronometro() {

                this.inicia = System.currentTimeMillis();

        }

        public long getInicia() {

                return inicia;

        }

        public long getFinaliza() {

                return finaliza;
```

```
        }
        public void inicia() {
    this.inicia = System.currentTimeMillis();
  }
        public void detener() {
    this.finaliza = System.currentTimeMillis();
  }
        public long lapsoDeTiempo() {
    return this.finaliza - this.inicia;
  }
}
```

CLASE PRINCIPAL

```
import java.util.Random;
public class Principal {
        public static void main(String[] args) {
                // TODO Auto-generated method stub
                Cronometro cr=new Cronometro();
                int[] n = new int[100000];
    Random r = new Random();
    for (int k = 0; k < n.length; k++) {
      n[k] = r.nextInt();
     //   System.out.print(n[k]+",");
    }
    cr.inicia();
    for (int k = 0; k < n.length - 1; k++) {
      int minIndex = k;
      for (int j = k + 1; j < n.length; j++) {
        if (n[j] < n[minIndex]) {
          minIndex = j;
```

```
            }
        }
        int temp = n[k];
        n[k] = n[minIndex];
        n[minIndex] = temp;
    }
    cr.detener();
    System.out.println("Tiempo: "+cr.lapsoDeTiempo()+" milisegundos");
        }
}
```

2. Algebra: ecuacion lineal 2 x 2. Disene la clase EcuacionLineal para un sistema de ecuaciones de 2X2

$$ax + by = e$$

$$cx + dy = f$$

$$x = \frac{ed - bf}{ad - bc} \qquad y = \frac{af - ec}{ad - bc}$$

La clase contiene:

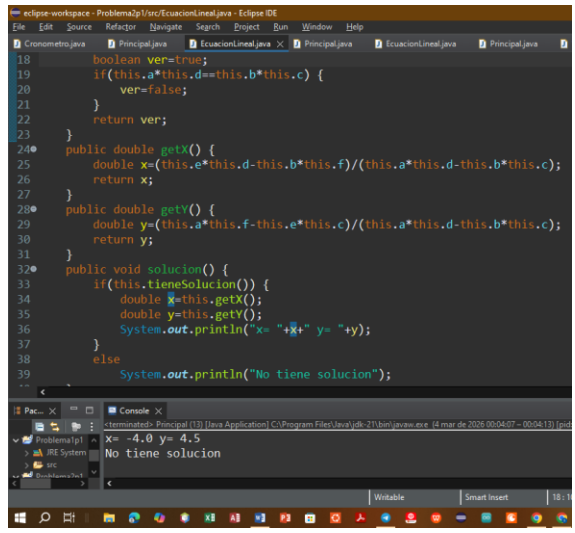
a) Los atributos privados a, b, c, d y f.
b) Un constructor para los argumentos para a, b, c, d, e y f.
c) Un método tieneSolucion() que devuelve true si ad bc no es cero.
d) Dos métodos getX() y getY() que retornan la solución de la ecuación.

Dibuje el diagrama UML de la clase e implementela. Escriba un programa de prueba (Test) que solicite al usuario ingresar a, b, c, d, e y f y muestre el resultado. Si ad bc es 0, indique que La ecuacion no tiene solucion .

EcuacionLineal
- a doublé - b doublé - c doublé - d doublé - e doublé - f doublé
+ EcuacionLineal(a,b,c,d,e,f) + tieneSolucion() + getX() + getY() + solucion()

CODIGO

CLASE ECUACION LINEAL




```
public class EcuacionLineal {
        private double a;
        private double b;
        private double c;
        private double d;
        private double e;
        private double f;
        public EcuacionLineal(double a, double b, double c, double d, double e, double f) {
                this.a = a;
                this.b = b;
                this.c = c;
                this.d = d;
                this.e = e;
                this.f = f;
        }
        public boolean  tieneSolucion() {
                boolean ver=true;
                if(this.a*this.d==this.b*this.c) {
                        ver=false;
                }
                return ver;
        }
        public double getX() {
                double x=(this.e*this.d-this.b*this.f)/(this.a*this.d-this.b*this.c);
                return x;
        }
```

```
	public double getY() {
		double y=(this.a*this.f-this.e*this.c)/(this.a*this.d-this.b*this.c);
		return y;
	}
	public void solucion() {
		if(this.tieneSolucion()) {
			double x=this.getX();
			double y=this.getY();
			System.out.println("x= "+x+" y= "+y);
		}
		else
			System.out.println("No tiene solucion");
	}
}
```

CLASE PRINCIPAL

```
public class Principal {
	public static void main(String[] args) {
		// TODO Auto-generated method stub
		EcuacionLineal el1=new EcuacionLineal(1,2,3,4,5,6);
		EcuacionLineal el2=new EcuacionLineal(-1,2,1,-2,5,6);
		//System.out.println(el1.tieneSolucion()+" "+el2.tieneSolucion());
		el1.solucion();
		el2.solucion();
	}
}
```

3. (Algebra: Ecuaciones Cuadráticas) Diseñe la clase EcuacionLineal para la ecuación cuadrática

$$Ax^2 + bx + c = 0.$$

La clase contiene:

a) Los atributos privados a, b y c, que representan a los 3 coeficientes.

b) Un constructor para los argumentos para a, b y c.
c) Un método getDiscriminante() que devuelve el discriminante b2 4ac.
d) Dos métodos getRaiz1() y getRaiz2() que retornan las dos raíces de la ecuación.

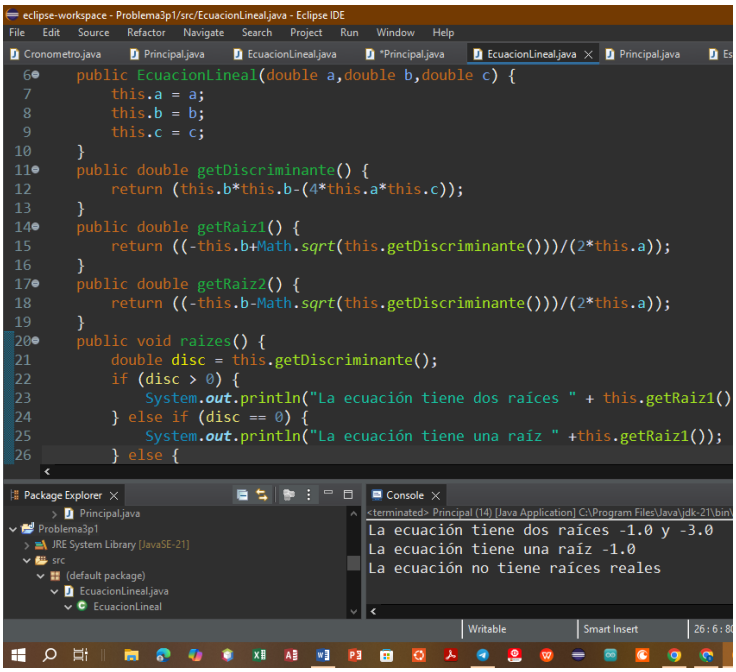
$$r1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \qquad r2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

Estos métodos solo son útiles si el discriminante es positivo. Si el discriminante es negativo, devuelva 0.

Dibuje el diagrama UML de la clase e impleméntela. Escriba un programa de prueba (Test) que solicite al usuario introducir los valores de a, b y c, y muestre el resultado según el discriminante. Si el discriminante es positivo, muestre las dos raíces. Si el discriminante es 0, muestre la única raíz. De lo contrario, muestre La ecuación no tiene raíces reales.

UML

EcuacionLineal
- a doublé - b doublé - c doublé
+ EcuacionLineal(a,b,c) + getDiscriminante() + getRaiz1() + getRaiz2() + raizes()



CODIGO

CLASE ECUACION LINEAL

```
public class EcuacionLineal {

        private double a;

        private double b;

        private double c;

        public EcuacionLineal(double a,double b,double c) {

                this.a = a;

                this.b = b;

                this.c = c;

        }

        public double getDiscriminante() {

                return (this.b*this.b-(4*this.a*this.c));
```

```
        }
        public double getRaiz1() {
                return ((-this.b+Math.sqrt(this.getDiscriminante()))/(2*this.a));
        }
        public double getRaiz2() {
                return ((-this.b-Math.sqrt(this.getDiscriminante()))/(2*this.a));
        }
        public void raizes() {
                double disc = this.getDiscriminante();
    if (disc > 0) {
      System.out.println("La ecuación tiene dos raíces " + this.getRaiz1() + " y " + this.getRaiz2());
    } else if (disc == 0) {
      System.out.println("La ecuación tiene una raíz " +this.getRaiz1());
    } else {
      System.out.println("La ecuación no tiene raíces reales");
    }
        }
}
```

CLASE PRINCIPAL

```
public class Principal {
        public static void main(String[] args) {
                // TODO Auto-generated method stub
                EcuacionLineal e1= new EcuacionLineal(1,4,3);
                EcuacionLineal e2= new EcuacionLineal(1,2,1);
                EcuacionLineal e3= new EcuacionLineal(1,2,3);
                e1.raizes();
                e2.raizes();
                e3.raizes();
        }
```

}

4. Estadísticas: calcular el promedio y la desviación estándar. En las aplicaciones comerciales, a menudo se le pide que calcule el promedio y la desviación estándar de los datos. La desviación estándar es una estadística que le indica cuan estrechamente se agrupan todos los diversos datos alrededor de la media en un conjunto de datos. Por ejemplo, ¿cuál es la edad promedio de los estudiantes en una clase? ¿Qué tan cerca están las edades? Si todos los estudiantes tienen la misma edad, la desviación es 0. Utilice la siguiente formula:

$$promedio = \frac{\sum_{i=1}^{n} x_i}{n} \qquad desviacion = \sqrt{\frac{\sum_{i=1}^{n}(x_i - promedio)^2}{n-1}}$$

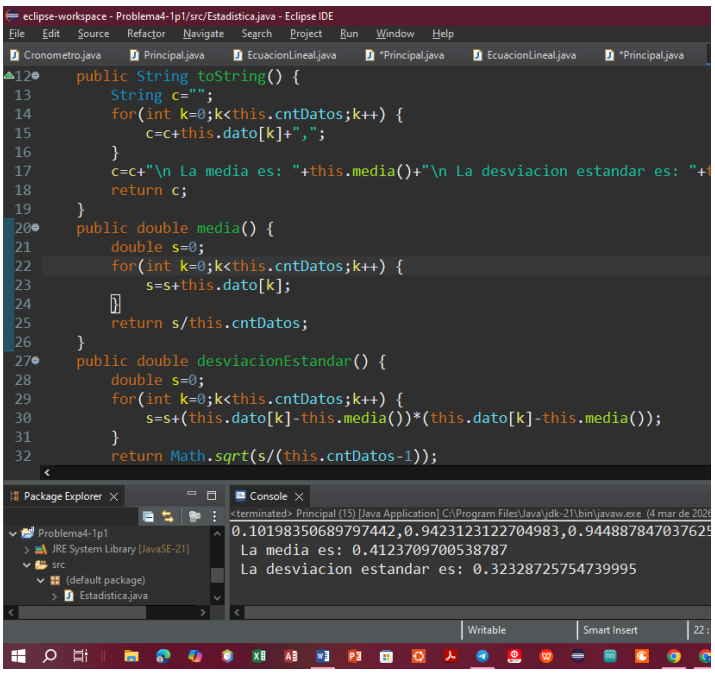
Su programa debe contener los siguientes métodos:

- promedio () obtiene el promedio de los valores en punto flotante
- desviación () devuelve la desviación estándar de los valores en punto flotante.

Escriba un programa que solicite al usuario ingresar 10 números, muestre el promedio y las desviaciones estándar de estos números.

Obtenga la solución en la Programación Modular-Estructura y luego en Programación Orientada a Objetos, para ello dibuje el diagrama UML para la clase Estadística y luego implemente la clase. Agregue de forma escrita las ventajas de utilizar la P.O.O.

Estadistica
- datos[50] doublé - cntDatos int
+ Estadistica(n) + toString() + media() + desviacionEstandar()



CODIGO ESTADISTICA

```
import java.util.Random;

public class Estadistica {

        private double dato[]=new double[50];

        public int cntDatos;

        public Estadistica(int n) {

                this.cntDatos = n;

                Random r = new Random();

                for(int k=0;k<this.cntDatos;k++) {
```

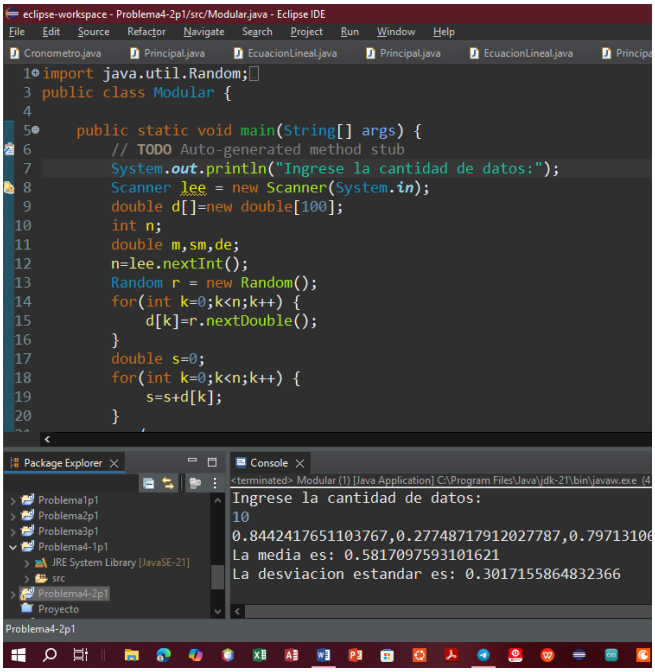
```
                this.dato[k]=r.nextDouble();
            }
        }
        public String toString() {
            String c="";
            for(int k=0;k<this.cntDatos;k++) {
                c=c+this.dato[k]+",";
            }
            c=c+"\n La media es: "+this.media()+"\n La desviacion estandar es:
"+this.desviacionEstandar();
            return c;
        }
        public double media() {
            double s=0;
            for(int k=0;k<this.cntDatos;k++) {
                s=s+this.dato[k];
            }
            return s/this.cntDatos;
        }
        public double desviacionEstandar() {
            double s=0;
            for(int k=0;k<this.cntDatos;k++) {
                s=s+(this.dato[k]-this.media())*(this.dato[k]-this.media());
            }
            return Math.sqrt(s/(this.cntDatos-1));
        }
}
```

CLASE PRINCIPAL

```
public class Principal {
```

```
        public static void main(String[] args) {
                // TODO Auto-generated method stub
                Estadistica e= new Estadistica(10);
                System.out.println(e.toString());
        }
        /*
         * Ventajas de usar poo
         * la ventaja de tener un programa mas ordenado y simplificado
         * la ventaja de usar los metodo las veces que queramos o que sean necesarias sin necesidad de copiar el codigo
         * la ventaja de tener menos lineas de codigo
        */
}
```

CODIGO MODULAR




```
import java.util.Random;
import java.util.Scanner;
public class Modular {
        public static void main(String[] args) {
                System.out.println("Ingrese la cantidad de datos:");
                Scanner lee = new Scanner(System.in);
                double d[]=new double[100];
                int n;
                double m,sm,de;
                n=lee.nextInt();
                Random r = new Random();
                for(int k=0;k<n;k++) {
                        d[k]=r.nextDouble();
```

```
		}
		double s=0;
		for(int k=0;k<n;k++) {
			s=s+d[k];
		}
		m=s/n;
		sm=0;
		for(int k=0;k<n;k++) {
			sm=sm+(d[k]-m)*(d[k]-m);
		}
		de=Math.sqrt(sm/(n-1));
		for(int k=0;k<n;k++) {
			System.out.print(d[k]+",");
		}
		System.out.println("\nLa media es: "+m);
		System.out.println("La desviacion estandar es: "+de);
	}
	/*
	 * Ventajas de usar POO
	 * la ventaja de tener un programa mas ordenado y simplificado
	 * la ventaja de usar los metodo las veces que queramos o que sean necesarias sin
necesidad de copiar el codigo
	 * la ventaja de tener menos lineas de codigo
	*/
}
```